

WALnut

Project Plan, Team Division & Phase 2 Evaluation Roadmap

Prepared for Team The Acid Mechanics — Vinayak Suyal, Sushmit Singh Rawat, Kanak Rawat, and Suhavi Jugran.

This document consolidates the project-planning discussion into a practical build plan. It deliberately uses completion phases and deliverables rather than an 8-week schedule.

Project overview

WALnut is a C++17 concurrent auction engine built from scratch. It uses custom page-based storage, a buffer pool, transactions with Strict Two-Phase Locking, wait-for-graph deadlock detection, Write-Ahead Logging (WAL) crash recovery, an HTTP API, and a dashboard.

The final demonstration should prove four things: data persists, concurrent bids remain correct, a real deadlock is detected and resolved, and committed data survives a controlled crash.

Phase 2 target

For a 70–75% evaluation, the system does not need every advanced feature polished. It must, however, have working code, tests, and a personal demo for every member.

- Custom storage can create, write, read, and reload auction records after restart.
- Buffer pool caches pages and reports basic hit/miss statistics.
- Concurrent placeBid operations are protected by exclusive item locks under Strict 2PL.
- WAL records are written and a basic crash-recovery path works.
- REST API and dashboard can place real bids and display core live activity.
- Deadlock detection can be limited to the deterministic two-transaction reserveBundle scenario at this stage.

Final team ownership

Vinayak — Storage manager and buffer pool

- Own the fixed-size page format, walnut.db file manager, metadata and page allocation.
- Implement users, items, and bids record persistence.
- Implement the BufferPool with fetch/pin/unpin/flush, dirty-page tracking, and Clock replacement.
- Expose hit/miss statistics and make the buffer pool thread-safe before concurrency integration.
- Write storage and buffer tests; assemble the architecture documentation.

Evaluation statement: “I implemented the page-based storage and buffer layer: raw file I/O, page allocation, persistent records, pin/unpin semantics, dirty-page write-back, and Clock replacement.”

Sushmit — Transactions, locking, and deadlock handling

- Own transaction lifecycle: BEGIN, COMMIT, ABORT/ROLLBACK.
- Implement exclusive item locks first, waiting queues, mutexes, and condition variables.
- Implement Strict 2PL so all write locks are held until commit or abort.
- Implement a thread pool for concurrent requests.
- Implement wait-for graph, cycle detection, victim selection, rollback, and retry for reserveBundle.

Evaluation statement: “I implemented the transaction manager, Strict 2PL lock manager, concurrent execution, and wait-for-graph deadlock detection with victim resolution.”

Kanak — WAL, recovery, and hash index

- Own walnut.wal format and append-only writer.
- Log BEGIN, UPDATE, COMMIT, ABORT, and optionally CHECKPOINT records.
- Implement the write-ahead rule: log must be durable before a dirty page can be flushed.
- Implement redo for committed transactions and undo for uncommitted transactions.
- Build fault-injection crash tests and a hash index on item_id.

Evaluation statement: “I implemented WAL durability, redo/undo recovery after crashes, controlled fault injection, and hash indexing for fast item lookup.”

Suhavi — REST API and dashboard

- Own the embedded HTTP/JSON API and its error handling.
- Build React + TypeScript bidding dashboard: items, highest bid, bid history, and manual bid form.
- Implement SSE telemetry for lock activity, WAL writes, and buffer events.
- Build a basic systems-monitor table first; advanced graphs are optional for Phase 2.
- Own API/dashboard tests, README run steps, API specification, and final demo flow.

Evaluation statement: “I implemented the REST/SSE communication layer and dashboard, so users can place real bids and observe engine activity in real time.”

Dependency order

Vinayak is the critical starting point because all actual records must be stored and read through the storage/buffer layer. Kanak and Sushmit can design and implement standalone components in parallel, but their complete integration depends on Vinayak’s stable interfaces. Suhavi can scaffold the API and UI early with mock data, then wire real endpoints after the engine layer is ready.

1. Vinayak: page format -> FileManager -> StorageManager -> basic BufferPool
2. Kanak: WAL record format/writer + hash index; integrate WAL flush hook after storage exists
3. Sushmit: transaction/lock manager + thread pool; integrate real placeBid after storage exists
4. Suhavi: REST/UI skeleton; connect real endpoints after transaction API works
5. Whole team: integration -> tests -> deadlock/recovery demos -> polish

Shared interfaces to freeze early

Create one shared header for the basic IDs, record types, and function signatures. Do not casually change these once teammates begin integration.

```
// Storage / Buffer (Vinayak)
Page& fetchPage(uint32_t page_id);
void pinPage(uint32_t page_id);
void unpinPage(uint32_t page_id, bool dirty);
void flushPage(uint32_t page_id);
Item getItem(const std::string& item_id);
uint64_t insertBid(const Bid& bid);
void updateItemAfterBid(const std::string& item_id, double amount,
                       const std::string& winner);

// Transactions / Locks (Sushmit)
TxnId beginTransaction();
void commitTransaction(TxnId txn_id);
void abortTransaction(TxnId txn_id);
LockResult acquireExclusive(TxnId txn_id, const std::string& item_id);
void releaseAllLocks(TxnId txn_id);

// WAL / Recovery (Kanak)
void logBegin(TxnId txn_id);
```

```

void logUpdate(const WalRecord& record);
void logCommit(TxnId txn_id);
void recover();

// API (Suhavi)
POST /users
POST /items
POST /items/{item_id}/bids
GET /items/{item_id}/highest-bid
GET /items/{item_id}/bids
POST /items/{item_id}/close
POST /reserve-bundle
GET /stream

```

Vinayak steps

1. Foundation

- Create CMake project, shared types, data directory, .gitignore, and storage headers.
- Set PAGE_SIZE to 4096.
- Implement PageHeader: page_id, page_type, record_count, and free-space offset.
- Use fixed-size records initially; do not start with fully variable-length slotted records.

2. File manager

- Create walnut.db.
- Implement readPage(page_id), writePage(page), allocatePage(type), and pageCount().
- Use disk offset = page_id × PAGE_SIZE.
- Ensure a written page can be read back byte-for-byte.

3. Storage manager

- Implement createUser, createItem, insertBid, getItem, getBids, and updateItemAfterBid.
- Keep IDs caller-supplied strings for the API; assign bid_id internally.
- Use simple allocation first; improve free-space handling later if needed.

4. Buffer pool

- Use a fixed number of frames, e.g. 8.
- Implement fetchPage, pinPage, unpinPage, flushPage, and getStats.
- Evict only unpinned pages; write dirty victims before replacement.
- Implement Clock replacement before considering LRU.

5. Required tests

- Write/read page round trip.
- Persist items and bids across restart.
- Pinned page cannot be evicted.
- Repeated page read produces a buffer hit.
- Dirty evicted page is persisted.

Sushmit steps

1. Transaction and lock types

- Define TxnState: ACTIVE, WAITING, COMMITTED, ABORTED.

- Define LockMode: EXCLUSIVE first; add shared locks only if required for reads.
- Create transaction IDs and transaction table.

2. Lock manager

- Create lock table: item_id -> holder(s), mode, waiting queue.
- Grant lock immediately when compatible.
- Otherwise enqueue request and block with a condition variable.
- On commit/abort, release locks and wake eligible waiters.

3. Transactional placeBid

```
BEGIN
Acquire exclusive lock on item
Read current bid through BufferPool
Reject if amount <= current_bid
Insert Bid record and update Item
Ask WAL manager to log changes
COMMIT
Release all locks
```

4. Concurrency tests

- Launch 10 threads with increasing valid bids on one item; final value must equal maximum.
- Hold a lock in one transaction and verify another waits.
- Release first transaction and verify the waiting transaction wakes.
- Verify invalid/low bid leaves item state unchanged.

5. Deadlock target

- Add wait-for edges whenever a transaction waits on another holder.
- Use DFS to find cycles.
- For Phase 2, support two-transaction reserveBundle(A,B) vs reserveBundle(B,A).
- Pick the youngest transaction as victim, abort it, release locks, then retry later.

Kanak steps

1. WAL format

- Create walnut.wal.
- Use records with LSN, txn_id, type, page_id, before image, and after image.
- Implement appendRecord, flush, readAllRecords, and current LSN.

2. Write-ahead integration

- Log BEGIN before updates.
- Log UPDATE before allowing the modified data page to flush.
- Log COMMIT when a transaction successfully finishes.
- Coordinate the flush-order hook with Vinayak.

3. Recovery

- Scan WAL at startup.
- Identify committed transactions.
- Redo after-images for committed changes.

- Undo before-images of transactions lacking COMMIT, in reverse order.
- A minimal recovery path is fine for Phase 2; document any simplifications honestly.

4. Fault injection and index

- Build a script/test that kills the engine at controlled transaction points and restarts it.
- Build hash index: item_id -> page_id, record_id using unordered_map.
- Start in-memory; persistence is optional after the basic demo works.

5. Required tests

- Log record exists before a data page is flushed.
- Committed bid survives SIGKILL/restart.
- In-flight/uncommitted bid is absent after restart.
- Index lookup returns the correct stored item location.

Suhavi steps

1. API server

- Create embedded HTTP server and connect to the engine service layer.
- Implement POST /users, POST /items, POST /items/{id}/bids.
- Implement GET highest-bid, GET bid history, POST close.
- Return validation, not-found, bid-too-low, auction-closed, and transaction-aborted errors.

2. Dashboard

- Create React + TypeScript + Vite project.
- Build active-items list, item detail/bid form, highest-bid display, and history.
- Manual bid must call the real POST bid endpoint—not a mock-only path.
- Use a simple selected fixed user_id roster (u1–u4).

3. SSE monitoring

- Implement GET /stream.
- Emit lock.acquired, lock.waiting, lock.released, wal.write, buffer.hit, buffer.miss.
- Show simple tables/counters in Phase 2; visual graph can come later.
- Add deadlock.detected/resolved when Sushmit's subsystem is ready.

4. Required tests

- Happy-path API tests for users/items/bids.
- Low bid returns 409; missing item returns 404; closed auction returns 409.
- Dashboard manual bid updates item and history.
- SSE observes at least lock and WAL events.

Final integration order

1. Vinayak hands stable StorageManager + BufferPool API to team.
2. Kanak attaches WAL-before-flush hooks and provides index lookup.
3. Sushmit routes transaction reads/writes through storage/buffer and calls WAL.
4. Suhavi connects REST endpoints to transaction service, then dashboard to REST/SSE.
5. Team verifies persistence -> concurrency -> recovery -> dashboard visibility.
6. Add reserveBundle deadlock demo and polish final presentation.

Phase 2 combined demo

- Vinayak: create items/bids, restart engine, prove persistence, show buffer stats.
- Sushmit: submit concurrent bids, show locking and correct final highest bid; demonstrate deadlock if ready.
- Kanak: show WAL records, controlled crash, restart recovery, and hash-index lookup.
- Suhavi: submit a bid through dashboard and show updated bid history plus live telemetry.

Definition of 70–75% complete

- All four members have meaningful committed code in their subsystem folders.
- Each member owns at least two or three tests that pass.
- A manual UI bid uses the same engine path as automated clients.
- Data persistence and concurrency are genuinely working—not mocked.
- WAL/recovery works for primary commit/in-flight cases, even if checkpointing is basic.
- Deadlock design is implemented or visibly partial; the deterministic scenario should be ready if possible.
- Documentation and a clean demo script explain any features that remain unfinished.

Git workflow

```
main
storage-vinayak
txn-locking-sushmit
wal-recovery-kanak
api-dashboard-suhavi
```

Rule: no component directly reads/writes walnut.db except StorageManager.
Rule: shared headers or record formats need review by affected subsystem owners.
Rule: each pull request includes a test or a reproducible demo command.

Presenter order

- Vinayak: architecture overview, storage pages, buffer pool, persistence.
- Sushmit: transaction lifecycle, Strict 2PL, concurrent bidding, deadlock logic.
- Kanak: WAL ordering, crash recovery, hash index.
- Suhavi: REST API, dashboard, SSE telemetry, integrated user flow.

Message to share with team

Team: Ignore the old week-by-week schedule. Our Phase 2 target is a demonstrable 70–75% complete system.

Build order:

1. Vinayak delivers stable storage + BufferPool APIs.
2. Kanak integrates WAL-before-flush and index support.
3. Sushmit integrates transactions/locks/concurrency over the storage layer.
4. Suhavi wires REST/SSE/dashboard to the real transaction service.

Every person must have: owned code, passing tests, a documentation section, and a live demo contribution. Use the subsystem boundaries in this document. Do not access walnut.db directly outside StorageManager.